Tutorial: Single-Agent Navigation

Goal Attraction + Sigmoid Obstacle Repulsion

Heterogeneous Swarm Robotics Education

This tutorial explains the complete Python program first, then walks through each section step by step in the same style used for introductory Arduino programming exercises.

1. Learning Objectives

After completing this tutorial, students should be able to explain how a single mobile agent can move toward a goal while avoiding an obstacle by combining two artificial-force components.

- Define the initial position, goal, obstacle, and sensing range.
- Generate 360 sensing directions around the agent.
- Use a logistic sigmoid to smoothly vary goal attraction.
- Use a decreasing logistic sigmoid to smoothly vary obstacle repulsion.
- Combine attraction and repulsion into one resultant motion vector.
- Update the agent position using a discrete-time simulation.
- Visualize the sensing range, obstacle detection, trajectory, and final path.

The key idea is that the agent does not follow a precomputed geometric path. At every simulation step, it computes a motion vector from the current situation.

```
Total force = Goal attraction force + Obstacle repulsion force
```

2. Complete Program

The complete program is shown below. The following sections explain the same code in smaller blocks. The program itself is not modified in the explanation.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

plt.close("all")

# ============================================================
# 1. SIMULATION PARAMETERS
# ============================================================

dt = 0.05
num_iterations = 4000

# Gains
Kg = 2.0                  # Goal attraction gain
Ko = 4.5                  # Obstacle repulsion gain

# Maximum velocity
MAX_SPEED = 5

# Goal tolerance
GOAL_TOLERANCE = 1.0


# ============================================================
# 2. ENVIRONMENT
# ============================================================

START = np.array([10.0, 10.0])
GOAL = np.array([90.0, 90.0])
OBSTACLE = np.array([52.0, 45.0])
OBSTACLE_RADIUS = 6.0


# ============================================================
# 3. SENSOR PARAMETERS
# ============================================================

SENSING_RADIUS = 22.0
NUM_SENSING_DIRECTIONS = 360


# ============================================================
# 4. SIGMOID PARAMETERS
# ============================================================

# Goal attraction sigmoid
GOAL_MID_DISTANCE = 12.0
GOAL_SLOPE = 0.25
```

```
# Obstacle repulsion sigmoid
REPULSION_MID_DISTANCE = 10.0
REPULSION_SLOPE = 0.50


# ============================================================
# 5. CREATE THE 360 SENSING DIRECTIONS
# ============================================================

angles = np.linspace(0, 2 * np.pi, NUM_SENSING_DIRECTIONS, endpoint=False)
sensing_directions = np.column_stack((np.cos(angles), np.sin(angles)))


# ============================================================
# 6. SIGMOID FUNCTIONS
# ============================================================

def sigmoid_goal_attraction(distance):
    """
    Attraction increases with distance.

    Far from goal -> approximately 1
    Near the goal -> approximately 0
    """
    exponent = -GOAL_SLOPE * (distance - GOAL_MID_DISTANCE)
    exponent = np.clip(exponent, -60, 60)
    return 1.0 / (1.0 + np.exp(exponent))


def sigmoid_obstacle_repulsion(distance):
    """
    True decreasing logistic sigmoid.

    distance = REPULSION_MID_DISTANCE gives exactly 0.5.

    Close to obstacle -> approximately 1
    Far from obstacle -> approximately 0
    """
    exponent = REPULSION_SLOPE * (distance - REPULSION_MID_DISTANCE)
    exponent = np.clip(exponent, -60, 60)
    return 1.0 / (1.0 + np.exp(exponent))


# ============================================================
# 7. GOAL ATTRACTION
# ============================================================

def calculate_goal_attraction(pos):

    # Vector from robot to goal
    vec = GOAL - pos
    distance = np.linalg.norm(vec)

    if distance < 0.0001:
        return np.zeros(2)

    # Direction toward goal
    direction = vec / distance

    # Sigmoid magnitude
    magnitude = sigmoid_goal_attraction(distance)

    return direction * magnitude


# ============================================================
# 8. OBSTACLE REPULSION
# ============================================================

def calculate_obstacle_repulsion(pos):
    """
    The robot senses the environment in 360 directions.

    If the obstacle is inside the sensing range, the direction
    that best points toward the obstacle is identified.

    A repulsive force is generated in the opposite direction.
    """

    # Vector robot -> obstacle
    vec_obstacle = OBSTACLE - pos
    distance_to_center = np.linalg.norm(vec_obstacle)
```

```
    if distance_to_center < 0.0001:
        return np.zeros(2), True, 0.0

    # Distance from robot to obstacle surface
    distance_to_surface = distance_to_center - OBSTACLE_RADIUS
    distance_to_surface = max(distance_to_surface, 0.0)

    # Is obstacle inside sensing range?
    if distance_to_surface > SENSING_RADIUS:
        return np.zeros(2), False, distance_to_surface

    # Direction from robot toward obstacle
    direction_to_obstacle = vec_obstacle / distance_to_center

    # Search the 360 sensing directions
    similarity = sensing_directions @ direction_to_obstacle
    index = np.argmax(similarity)
    detected_direction = sensing_directions[index]

    # Sigmoid obstacle repulsion
    magnitude = sigmoid_obstacle_repulsion(distance_to_surface)

    # Repulsion is opposite to detected obstacle direction
    repulsion = -detected_direction * magnitude

    return repulsion, True, distance_to_surface


# ============================================================
# 9. SIMULATION
# ============================================================

position = START.copy()

positions_history = [position.copy()]
detected_history = [False]
repulsion_history = [0.0]
goal_force_history = [0.0]

for t in range(num_iterations):

    # Goal attraction
    fg = Kg * calculate_goal_attraction(position)

    # Obstacle repulsion
    fo_raw, detected, obstacle_distance = calculate_obstacle_repulsion(position)
    fo = Ko * fo_raw

    # Total force
    force = fg + fo

    # Limit maximum velocity
    speed = np.linalg.norm(force)

    if speed > MAX_SPEED:
        force = force / speed * MAX_SPEED

    # Update position
    position = position + force * dt

    # Save history
    positions_history.append(position.copy())
    detected_history.append(detected)
    repulsion_history.append(np.linalg.norm(fo))
    goal_force_history.append(np.linalg.norm(fg))

    # Goal reached?
    distance_goal = np.linalg.norm(GOAL - position)

    if distance_goal < GOAL_TOLERANCE:
        print("\nGOAL REACHED")
        print("Iteration:", t)
        print("Position:", position)
        break

positions_history = np.array(positions_history)


# ============================================================
# 10. ANIMATION
# ============================================================

fig, ax = plt.subplots(figsize=(8, 8))
```

```
ax.set_xlim(0, 100)
ax.set_ylim(0, 100)
ax.set_aspect("equal", adjustable="box")
ax.grid(True)
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_title("Single Agent Navigation\nGoal Attraction + Sigmoid Obstacle Repulsion")


# ------------------------------------------------------------
# Draw Goal
# ------------------------------------------------------------

ax.scatter(GOAL[0], GOAL[1], marker="X", s=200, label="Goal")


# ------------------------------------------------------------
# Draw Start
# ------------------------------------------------------------

ax.scatter(START[0], START[1], marker="o", s=100, label="Start")


# ------------------------------------------------------------
# Draw Obstacle
# ------------------------------------------------------------

obstacle_circle = plt.Circle(OBSTACLE, OBSTACLE_RADIUS, alpha=0.7, label="Obstacle")
ax.add_patch(obstacle_circle)


# ============================================================
# AGENT
# ============================================================

agent, = ax.plot([], [], "o", markersize=10, label="Agent")


# ============================================================
# PATH
# ============================================================

path_line, = ax.plot([], [], linewidth=2, label="Path")


# ============================================================
# SENSING RANGE
# ============================================================

sensing_circle = plt.Circle(START, SENSING_RADIUS, fill=False, linestyle="--", linewidth=1.5)
ax.add_patch(sensing_circle)


# ============================================================
# DETECTION LINE
# ============================================================

detection_line, = ax.plot([], [], linestyle="--", linewidth=1)


# ============================================================
# STATUS TEXT
# ============================================================

status_text = ax.text(2, 97, "", fontsize=11, verticalalignment="top")

ax.legend(loc="lower right")

# Show every simulation step
ANIMATION_SKIP = 1


# ============================================================
# 11. ANIMATION UPDATE
# ============================================================

def update(frame):

    pos = positions_history[frame]

    # Agent position
    agent.set_data([pos[0]], [pos[1]])

    # Path
```

```
    path_line.set_data(positions_history[:frame + 1, 0], positions_history[:frame + 1, 1])

    # Move sensing circle
    sensing_circle.center = (pos[0], pos[1])

    # Obstacle detection visualization
    if detected_history[frame]:
        detection_line.set_data([pos[0], OBSTACLE[0]], [pos[1], OBSTACLE[1]])
        status = "Obstacle detected: YES\n" f"Repulsion force: " f"{repulsion_history[frame]:.2f}"
    else:
        detection_line.set_data([], [])
        status = "Obstacle detected: NO\n" f"Repulsion force: " f"{repulsion_history[frame]:.2f}"

    status_text.set_text(status)

    return agent, path_line, sensing_circle, detection_line, status_text


# ============================================================
# 12. CREATE ANIMATION
# ============================================================

ani = FuncAnimation(fig, update, frames=range(0, len(positions_history), ANIMATION_SKIP), interval=20, repeat=False)


# ============================================================
# 13. FINAL PATH
# ============================================================

fig2, ax2 = plt.subplots(figsize=(8, 8))

ax2.set_xlim(0, 100)
ax2.set_ylim(0, 100)
ax2.set_aspect("equal", adjustable="box")
ax2.grid(True)
ax2.set_xlabel("X")
ax2.set_ylabel("Y")
ax2.set_title("Final Agent Path")


# Path
ax2.plot(positions_history[:, 0], positions_history[:, 1], linewidth=2, label="Path")

# Start
ax2.scatter(START[0], START[1], s=100, label="Start")

# Goal
ax2.scatter(GOAL[0], GOAL[1], marker="X", s=200, label="Goal")

# Obstacle
obstacle_final = plt.Circle(OBSTACLE, OBSTACLE_RADIUS, alpha=0.7)
ax2.add_patch(obstacle_final)

# Show the obstacle sensing boundary for reference in the final plot
obstacle_detection_zone = plt.Circle(OBSTACLE, OBSTACLE_RADIUS + SENSING_RADIUS, fill=False, linestyle=":",
linewidth=1)
ax2.add_patch(obstacle_detection_zone)

ax2.legend()

plt.show()
```

3. Importing the Required Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

plt.close("all")
```

- NumPy provides vectors, distances, trigonometric functions, and array operations.
- Matplotlib creates the figures that show the environment and final trajectory.
- FuncAnimation updates the agent position frame by frame to create the animated simulation.
- plt.close("all") closes any previously opened Matplotlib figures before a new run starts.

4. Simulation Parameters

```
dt = 0.05
num_iterations = 4000
Kg = 2.0
Ko = 4.5
MAX_SPEED = 5
GOAL_TOLERANCE = 1.0
```

- dt is the simulation time step. The agent position is updated once every dt units of simulated time.
- num_iterations limits how many updates are allowed if the goal is not reached earlier.
- Kg scales the strength of the goal-attraction force.
- Ko scales the strength of the obstacle-repulsion force.
- MAX_SPEED prevents the resultant motion vector from becoming excessively large.
- GOAL_TOLERANCE defines how close the agent must be to the goal before the simulation declares success.

5. Environment Definition

```
START = np.array([10.0, 10.0])
GOAL = np.array([90.0, 90.0])
OBSTACLE = np.array([52.0, 45.0])
OBSTACLE_RADIUS = 6.0
```

- START is the initial XY position of the agent.
- GOAL is the target XY position.
- OBSTACLE is the XY position of the center of the circular obstacle.
- OBSTACLE_RADIUS defines the physical size of the obstacle. The repulsion calculation uses distance to the obstacle surface, not only distance to its center.

6. Sensor Parameters and the 360-Degree Sensing Model

```
SENSING_RADIUS = 22.0
NUM_SENSING_DIRECTIONS = 360

angles = np.linspace(0, 2 * np.pi, NUM_SENSING_DIRECTIONS, endpoint=False)
sensing_directions = np.column_stack((np.cos(angles), np.sin(angles)))
```

- SENSING_RADIUS defines the maximum distance at which the obstacle is considered detectable.
- NUM_SENSING_DIRECTIONS = 360 creates one sensing direction per degree around the agent.
- np.linspace creates 360 equally spaced angles from 0 to 2π radians.
- cos(angle) and sin(angle) convert each angle into a 2-D unit vector.
- The final sensing_directions array therefore contains 360 direction vectors surrounding the agent.

7. Sigmoid Parameters

```
GOAL_MID_DISTANCE = 12.0
GOAL_SLOPE = 0.25
REPULSION_MID_DISTANCE = 10.0
REPULSION_SLOPE = 0.50
```

- Each sigmoid is controlled by a midpoint and a slope.
- At the midpoint, the corresponding sigmoid value is exactly 0.5.
- The slope determines how rapidly the force transitions around the midpoint.
- For obstacle repulsion, a larger slope creates a sharper change between weak and strong repulsion.

8. Goal-Attraction Sigmoid

```
def sigmoid_goal_attraction(distance):
    exponent = -GOAL_SLOPE * (distance - GOAL_MID_DISTANCE)
    exponent = np.clip(exponent, -60, 60)
    return 1.0 / (1.0 + np.exp(exponent))
```

- This is a logistic sigmoid whose value increases as the agent gets farther from the goal.
- When distance equals GOAL_MID_DISTANCE, the sigmoid value is 0.5.
- Far from the goal, the value approaches 1, which produces strong attraction.
- Near the goal, the value approaches 0, which reduces the attraction smoothly.
- np.clip limits the exponent and avoids numerical overflow in exp().

9. Obstacle-Repulsion Sigmoid

```
def sigmoid_obstacle_repulsion(distance):
    exponent = REPULSION_SLOPE * (distance - REPULSION_MID_DISTANCE)
    exponent = np.clip(exponent, -60, 60)
    return 1.0 / (1.0 + np.exp(exponent))
```

- This is a decreasing logistic sigmoid.
- When the agent is close to the obstacle, the sigmoid approaches 1 and repulsion is strong.
- When the agent is far from the obstacle, the sigmoid approaches 0 and repulsion becomes weak.
- At REPULSION_MID_DISTANCE, the sigmoid is exactly 0.5.

10. Calculating Goal Attraction

```
def calculate_goal_attraction(pos):
    vec = GOAL - pos
    distance = np.linalg.norm(vec)
    if distance < 0.0001:
        return np.zeros(2)
    direction = vec / distance
    magnitude = sigmoid_goal_attraction(distance)
    return direction * magnitude
```

- GOAL - pos gives the vector pointing from the agent toward the goal.
- np.linalg.norm(vec) calculates the Euclidean distance to the goal.
- Dividing vec by its magnitude produces a unit vector that contains only direction.
- The sigmoid supplies the force magnitude.
- Multiplying direction by magnitude produces the final attraction vector.

11. Calculating Obstacle Repulsion

```
vec_obstacle = OBSTACLE - pos
distance_to_center = np.linalg.norm(vec_obstacle)
distance_to_surface = distance_to_center - OBSTACLE_RADIUS
if distance_to_surface > SENSING_RADIUS:
    return np.zeros(2), False, distance_to_surface
```

- The program first computes the vector and distance from the agent to the obstacle center.
- It then subtracts OBSTACLE_RADIUS to obtain the distance to the physical obstacle surface.
- If that surface is farther away than SENSING_RADIUS, no obstacle force is generated.
- The function also returns a Boolean value indicating whether the obstacle is currently detected.

12. Selecting the Best of the 360 Sensing Directions

```
direction_to_obstacle = vec_obstacle / distance_to_center
similarity = sensing_directions @ direction_to_obstacle
index = np.argmax(similarity)
detected_direction = sensing_directions[index]
```

- direction_to_obstacle is the exact unit vector from the agent toward the obstacle.
- The @ operator computes the dot product between that direction and all 360 sensing vectors.
- A dot product closest to 1 means the sensing vector points most nearly toward the obstacle.
- np.argmax selects the index of the best matching sensing direction.
- The resulting detected_direction is therefore the one-degree sensing direction that best points toward the obstacle.

13. Creating the Repulsive Force

```
magnitude = sigmoid_obstacle_repulsion(distance_to_surface)
repulsion = -detected_direction * magnitude
```

- The sigmoid determines how strong the repulsion should be based on distance to the obstacle surface.
- detected_direction points toward the obstacle.
- The negative sign reverses that direction, so the force points away from the obstacle.
- This raw vector is later multiplied by Ko inside the main simulation loop.

14. Main Simulation Loop

```
fg = Kg * calculate_goal_attraction(position)
fo_raw, detected, obstacle_distance = calculate_obstacle_repulsion(position)
fo = Ko * fo_raw
force = fg + fo
```

- fg is the scaled goal-attraction force.
- fo is the scaled obstacle-repulsion force.
- The total force is simply the vector sum of the two components.
- When no obstacle is detected, fo becomes zero and the agent moves only toward the goal.
- When the obstacle enters the sensing region, repulsion changes the resultant direction and bends the path around the obstacle.

15. Limiting Speed and Updating Position

```
speed = np.linalg.norm(force)
if speed > MAX_SPEED:
    force = force / speed * MAX_SPEED

position = position + force * dt
```

- The first block limits the magnitude of the total force while preserving its direction.
- The second line performs a simple Euler update.
- Conceptually, the program treats the force vector as a commanded planar velocity and advances the point-agent by force × dt each iteration.

16. Saving the History and Detecting the Goal

```
positions_history.append(position.copy())
detected_history.append(detected)
repulsion_history.append(np.linalg.norm(fo))
goal_force_history.append(np.linalg.norm(fg))

distance_goal = np.linalg.norm(GOAL - position)
if distance_goal < GOAL_TOLERANCE:
    break
```

- The history arrays store the information required later for plotting and animation.
- positions_history records the complete agent trajectory.
- detected_history records whether the obstacle was sensed at each step.
- repulsion_history and goal_force_history store force magnitudes.
- The loop stops as soon as the agent is within GOAL_TOLERANCE of the goal.

17. Animation and Sensing-Range Visualization

```
sensing_circle = plt.Circle(START, SENSING_RADIUS, fill=False, linestyle="--",
linewidth=1.5)
ax.add_patch(sensing_circle)
...
sensing_circle.center = (pos[0], pos[1])
```

- The dashed circle represents the current sensing radius of the agent.
- Its center is updated every animation frame so that it moves together with the agent.
- Visually, students can see exactly when the obstacle enters or leaves the sensing region.

18. Detection Line and Status Display

```
if detected_history[frame]:
    detection_line.set_data([pos[0], OBSTACLE[0]], [pos[1], OBSTACLE[1]])
    status = "Obstacle detected: YES\n" f"Repulsion force: "
f"{repulsion_history[frame]:.2f}"
else:
    detection_line.set_data([], [])
```

- When the obstacle is detected, a dashed line is drawn from the agent to the obstacle center.
- The text display changes to Obstacle detected: YES and shows the current repulsion-force magnitude.
- When the obstacle is outside the sensing range, the detection line disappears.

19. Final Path Plot

```
ax2.plot(positions_history[:, 0], positions_history[:, 1], linewidth=2, label="Path")
...
obstacle_detection_zone = plt.Circle(OBSTACLE, OBSTACLE_RADIUS + SENSING_RADIUS,
fill=False, linestyle=":", linewidth=1)
```

- The second figure shows the complete path after the simulation has finished.
- The start, goal, physical obstacle, and path are displayed together.
- The dotted circle around the obstacle provides a reference for the region in which the obstacle surface can enter the agent sensing range.

20. Overall Navigation Sequence

During every iteration, the agent repeats the same control sequence:

- Measure the current vector and distance to the goal.
- Calculate the goal-attraction vector.
- Check whether the obstacle surface is inside the sensing radius.
- If detected, select the best of 360 sensing directions.
- Calculate the sigmoid obstacle-repulsion vector.
- Add the attraction and repulsion vectors.
- Limit the resultant magnitude if necessary.
- Update the XY position.
- Repeat until the goal is reached.

```
Position(k+1) = Position(k) + [F_goal(k) + F_obstacle(k)] * dt
```

21. Suggested Student Experiments

A useful way to understand the controller is to change one parameter at a time and observe how the path changes.

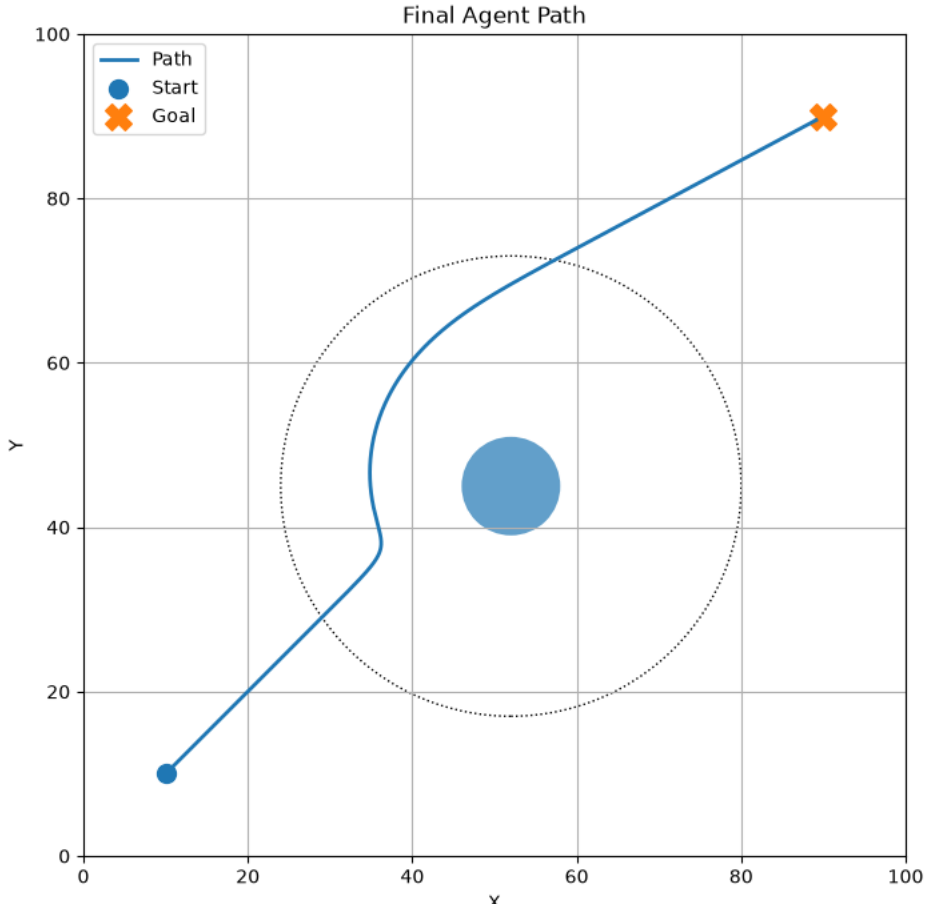
Parameter	Experiment
Ko	Increase or decrease the obstacle-repulsion gain and observe the clearance around the obstacle.
Kg	Change the goal-attraction gain and observe how strongly the agent attempts to return to the direct goal direction.
SENSING_RADIUS	Change how early the obstacle becomes detectable.
REPULSION_MID_DISTANCE	Move the 0.5 point of the obstacle sigmoid closer to or farther from the obstacle.
REPULSION_SLOPE	Make the obstacle response more gradual or more abrupt.
GOAL_SLOPE	Change how gradually the attraction weakens near the goal.

START / GOAL / OBSTACLE	Move the geometry and observe when the potential-field combination succeeds or produces a different path.

22. Connection to the Physical Spider Robot

This simulation treats the agent as a point that can move directly in any XY direction. The physical Spider robot cannot do that directly because it must rotate and move forward using discrete motion commands. The next stage is therefore to convert the resultant navigation vector into a desired heading and then map heading error into the Spider commands already validated in the ground-swarm platform.

```
Resultant force vector -> desired heading -> heading error -> turn / forward command
```



\